VERIGATE

Control the authority.
Not just the transaction.

A policy-to-execution gate for AI-agent payments on PortalDot V3 EVM and native Substrate testnets.

EVM GUARD

NATIVE SUBSTRATE

SIGNED AUTHORIZATION

THE RULE

An agent can propose a payment.

It cannot authorize **itself to execute it.**

RINAT / SOLO BUILDER

Wallet access is not spending authority

The missing control is between an agent’s request and the side effect.

01 / AGENT ACCESS

The agent can construct a valid payment request and call a wallet or payment interface.

Technical access is not policy approval.

02 / POLICY GAP

A valid transaction can still be outside the user’s intent: wrong recipient, excessive amount or unexpected timing.

The execution layer needs a separate authority decision.

03 / VERIGATE

Evaluate the proposed action against deterministic rules.

Issue a signed, action-bound decision before execution.

Reject mismatches at the gate.

THE DESIGN PRINCIPLE

Separate the ability to request an action from the authority to execute it.

A realistic failure path

A legitimate agent interface can still produce an unauthorized payment.

PROMPT

Pay the supplier

>

AGENT

Chooses recipient + amount

>

WALLET

Signs a valid transaction

>

SIDE EFFECT

Funds leave the account

INSERT AN AUTHORITY CHECK BEFORE THE SIDE EFFECT

The gate binds approval to the exact target, amount, network and action. A different action needs a new decision.

Without an enforcement point, a policy decision is only advice.

Two execution paths. One authorization concept.

The adapters are separate. Their enforcement guarantees are not interchangeable.

AGENT
typed intent

VERIGATE
policy + Ed25519

BRIDGE / ADAPTER
verify + bind

EXECUTION
path-specific checks

EVM PATH

Bridge verifies Ed25519 off-chain, then creates EIP-712 authorization. Guard verifies EIP-712 and enforces rules against its own contract balance.

NATIVE SUBSTRATE PATH

Adapter verifies policy before local signing / submit. Runtime does not verify Verigate's signature. Direct transactions outside this adapter bypass its checks.

EVM: signed decision becomes scoped authorization

The deployed guard is the final on-chain enforcement point for its own balance.

1. **Policy**
   Verigate returns ALLOW + signed artifacts.
2. **Verify**
   Bridge checks Ed25519 and exact action off-chain.
3. **Authorize**
   Dedicated signer creates EIP-712 authorization.
4. **Enforce**
   Guard checks signer, nonce, expiry, recipient and caps.

DEPLOYED V2 POLICY

0.1 POT

maximum per transfer

0.5 POT

per resettable 3,600-second interval

RECIPIENT ALLOWLIST

Not a global wallet firewall: only funds routed through the guard contract are controlled.

A real 0.1 POT transfer reached PortalDot V3

Recorded V2 execution evidence: successful receipt and one matching TransferExecuted event.

SUCCESSFUL TESTNET TRANSFER

0.1 POT

RECEIPT STATUS 1

1 MATCHING EVENT

CHAIN ID

420420777

BLOCK

149241

CONTRACT

0x87C4...E046C6

TRANSACTION HASH

0xaf3df785fdeaf85bbc9af2ae1342a7c664f4a55877469c4996eff2fe90f5551e

Re-queried through one configured PortalDot RPC. Testnet evidence, not independent RPC authentication.

Record: docs/evidence/portaldot-v2-live-transfer-2026-10-22.json

The guard rejects invalid payment parameters

Read-only eth_call probes exercised the deployed contract’s revert paths.

UNLISTED RECIPIENT

Probe: recipient not present in the configured allowlist.

Observed result:
InvalidRecipient()

OVER-CAP AMOUNT

Probe: amount one wei above the 0.1 POT per-transfer cap.

Observed result:
AmountExceedsCap(amount, cap)

EVIDENCE BOUNDARY

Read-only simulations, not mined reverted transactions. No transaction was broadcast and no fee was paid.

Local Hardhat tests additionally cover interval accounting and nonce behavior.

Native Substrate: a separate adapter completed a transfer

This path demonstrates pre-submit policy enforcement, not runtime-level Verigate signature verification.

01 / POLICY

Verigate ALLOW recorded.
Adapter checked the signed decision and action before submission.

02 / DRY RUN

system_dryRun succeeded before broadcast.

Balances.transfer_keep_alive was then submitted.

03 / FINALIZED

One matching balances.Transfer event and TransactionFeePaid.

Re-query located block 175489.

EXTRINSIC

0x098c3b320fd38a3062a5305b391fae81026aed267e5ed847186578a13600ef02

Fee recorded: 0.01490531479031 POT | Runtime: portaldot-v3/3002

One configured RPC; no second-provider check; saved evidence lacks original Ed25519 artifacts.

What is protected - and what is not

This is a pair of scoped adapters, not universal transaction interception.

PROTECTED IN THE DEMO

- **EVM guard balance**
  Transfers routed through the deployed V2 contract.
- **Native adapter path**
  Transactions submitted through the policy-checking adapter.
- **Exact action binding**
  Recipient, amount, nonce, deadline and action ID.

NOT PROTECTED / NOT CLAIMED

- **External wallet activity**
  Direct transactions outside the adapter.
- **Other contract balances**
  EVM guard cannot control funds it does not hold.
- **Production assurances**
  No audit, mainnet readiness or Wallet/DaaS integration.

Substrate runtime does not verify Verigate’s Ed25519 signature. The adapter does.

EVM contract verifies EIP-712 authorization; bridge verifies Ed25519 off-chain.

Why this is useful to PortalDot developers

A reference implementation for agent-driven payment flows, not a claim of ecosystem adoption.

REUSABLE PATTERN

- Typed payment intent
- Signed policy decision
- Exact action binding
- Fail-closed adapter checks
- Explicit submission boundary

PORTALDOT COVERAGE

- V3 EVM guard contract
- Native Substrate adapter
- Read-only preflight + dry-run
- Machine-readable evidence
- Runnable tests and guides

WHAT A TEAM CAN TEST

- Reproduce a payment flow
- Adapt policy to app rules
- Inspect evidence boundaries
- Measure integration effort
- Decide whether to pilot

Developer value is a hypothesis until an independent PortalDot team reproduces or adopts the flow.

No PortalDot Wallet or DaaS integration, team endorsement, or external adoption is claimed.

Commercial direction: plausible, not yet validated

The product hypothesis is infrastructure for teams that let software initiate payments.

HOSTED SERVICE

Policy evaluation API

Signed authorization receipts

Usage-based or tiered pricing

SELF-HOSTED

Run policy and evidence stack

Keep policy data in customer infrastructure

Enterprise support

INTEGRATION WORK

Wallet / payment adapters

Policy design and deployment

Security and operational review

CURRENT COMMERCIAL STATUS

No paying customers, revenue, production users or measured demand are claimed.

Next proof: one independent developer reproduces the integration and confirms a real use case.

Built and demonstrated. Next: independent validation.

A testnet prototype with explicit limits and public implementation evidence.

BUILT

- Deployed PortalDotAgentGuardV2
- Successful 0.1 POT EVM transfer
- Read-only rejection probes
- Separate native Substrate transfer
- Local tests, docs and evidence

NEXT

- Second-provider evidence checks
- Independent developer reproduction
- Wallet / DaaS integration discovery
- Threat model + external security audit
- Pilot with measurable acceptance criteria

LIVE DEMO

rudimentall1.github.io/verigate-portaldot/?guard=0x87C494a5e4B0A8E055F283767579667E10E046C6

SOURCE

github.com/rudimentall1/verigate-portaldot

Rinat / Solo Builder / PortalDot Hacker House 2026